MLPerf Storage

Vector Database Benchmark

Reference Guide

Version: MLPerf Storage v3.0.0
License: Apache 2.0

MLCommons Storage Working Group — VDB Task Group
Milvus + DiskANN / HNSW / AISAQ

1. What This Benchmark Does

Vector databases are the backbone of modern AI retrieval systems. They store high-dimensional embedding vectors — dense numerical representations of text, images, code, and other data — and perform approximate nearest neighbor (ANN) search to find similar items at millisecond latency across billions of vectors.

This benchmark simulates the storage I/O patterns generated by production vector search workloads against Milvus (the most widely deployed open-source vector database, with ~25K GitHub stars and ~700K monthly Docker pulls as of 2025). It measures how your storage subsystem performs under controlled, reproducible conditions by varying four orthogonal stress axes:

- **Vector Dimension (512–1536):** Controls per-vector byte size and I/O amplification
- **Indexing Algorithm (DiskANN / HNSW / AISAQ):** Determines whether search pressure falls on SSD or RAM
- **Dataset Scale (100K–10M vectors):** Sets the working set size relative to available memory
- **Query Concurrency (1–16 processes):** Controls parallel I/O pressure on storage

The benchmark answers questions that matter for storage infrastructure decisions:

- How many IOPS does your NVMe SSD deliver under realistic ANN search patterns?
- At what dataset scale does your working set exceed page cache, forcing cold SSD reads?
- What is the recall vs. storage throughput tradeoff for DiskANN at different search_list values?
- How does HNSW (RAM-resident) compare to DiskANN (SSD-resident) on your hardware?

This is not a pass/fail test. It is a diagnostic tool for storage architects, SSD vendors, and performance engineers working on AI infrastructure.

> *Note: Unlike synthetic fio workloads, this benchmark generates the actual random read I/O patterns produced by graph-based ANN search through a real Milvus instance, capturing the true interaction between vector index traversal, Milvus segment management, MinIO object storage, and the Linux page cache.*

2. Why This Workload Matters — Real-World Context

Vector database workloads are among the fastest-growing storage consumers in enterprise AI infrastructure. Understanding their I/O characteristics is critical for storage sizing, SSD selection, and system architecture decisions.

2.1 Market and Adoption

The vector database market was valued at approximately $2.2 billion in 2024, with projections exceeding $10 billion by 2032 at a 21%+ CAGR. Milvus alone serves ~700K monthly Docker pulls, and Pinecone has raised over $130 million. Every major cloud provider now offers managed vector database services.

2.2 Production Use Cases Driving Storage Demand

Use Case	Typical Scale	Dimension	I/O Pattern	Example
RAG for Enterprise LLMs	10M–1B+ chunks	768–1536	Read-heavy, random 4–64 KB	ChatGPT Enterprise, Claude with internal docs
E-commerce Product Search	100M–500M products	256–512	Read-heavy, bursty	Amazon product recs, Shopify vector search
Content Moderation	1B+ embeddings	512–768	Read-heavy, sustained	Trust & safety scanning uploads
Fraud Detection	10M–100M signals	128–512	Write-heavy ingest, read-heavy query	Fintech real-time fraud scoring
Code Search / Copilot	50M–500M chunks	768–1536	Read-heavy, multi-turn	GitHub Copilot, Cursor, internal code search
Multi-Modal Search	10M–100M images	512–2048	Read-heavy, large vectors	CLIP-based image search, audio fingerprinting
Autonomous Driving Maps	100M+ map tiles	128–256	Read-heavy, geospatial	Tesla, Waymo map tile similarity

2.3 Why Storage Is the Bottleneck

When vector datasets exceed available RAM (the norm at production scale), the ANN index must be served from SSD. A 2025 IISWC paper from TU Delft on storage-based vector database I/O characterization found that Milvus with DiskANN generates stable but heavy random read I/O during search, and that SSDs are not saturated under typical configurations — meaning there is room to improve storage utilization with proper tuning.

- DiskANN dominates storage I/O: The SSD-resident graph index generates 4–64 KB random reads per hop during graph traversal. Higher search_list values increase hops per query proportionally.
- HNSW is RAM-bound: Once loaded, HNSW generates negligible disk I/O — making it useful as a baseline for isolating storage overhead.
- Cache hit ratio matters enormously: A 2025 VLDB paper on pgvector SSD performance found that HNSW Layer 0 exhibits low cache hit ratios, making it the dominant factor in I/O overhead. io_uring-based async I/O can significantly reduce buffer cache misses.

- Per-query read bandwidth decreases with concurrency: At concurrency 256 vs. 1, per-query bandwidth drops 9.5–13.4% due to SSD queue contention — but aggregate throughput increases.
- Tiered storage is the industry pattern: Dell's 2024 VDB infrastructure white paper recommends pinning hot indices on NVMe and offloading long-tail vectors to cheaper block storage.

2.4 How This Benchmark Connects to Real Workloads

Benchmark Configuration	Real-World Equivalent
1M × 1536d DiskANN, 4 procs, warm cache	Small RAG deployment serving ~50 concurrent users
10M × 1536d DiskANN, 8 procs, cold cache	Enterprise RAG backend after service restart (cold page cache)
10M × 1536d DiskANN, 16 procs, sl=800+	Peak-hour load on production embedding search service
1M × 512d AISAQ, 4 procs, both caches	Memory-constrained edge deployment with compressed index
10M × 1536d HNSW, 8 procs, warm	RAM-rich cloud deployment (256+ GB instances)

3. Architecture Overview

The benchmark implements a three-phase workflow that generates distinct storage I/O patterns at each stage.

Phase 1 — Load Vectors (load_vdb.py): Write-heavy. Generates synthetic float32 vectors with configurable distribution (uniform, normal, or zipfian) and inserts them into Milvus collections in batched chunks. Triggers index building and optional compaction.

Phase 2 — Compact & Index (compact_and_watch.py): Background I/O. Monitors segment compaction and index build progress. Compaction consolidates small segments into larger ones, critical for stable I/O patterns during benchmarking.

Phase 3 — Benchmark Query (enhanced_bench.py): Read-heavy. The unified benchmark tool with two execution paths: Path A (runtime/query-count mode with per-worker CSV, P99.9/P99.99 stats, FLAT GT auto-creation) and Path B (parameter sweep across search_list/ef, warm/cold/both cache, optional RSS budget and host memory guards).

Recall Measurement: Both paths compute recall@k against a brute-force FLAT index (exact nearest neighbors), entirely outside the timed benchmark loop to avoid inflating latency numbers.

Disk I/O Measurement: Both paths diff /proc/diskstats before and after the benchmark to capture per-device read/write bytes, IOPS, and throughput (MB/s), filtering out virtual/loop devices.

4. Project Structure

Module	Purpose
`load_vdb.py`	Generates float32 vectors (uniform/normal/zipfian), creates Milvus collections, inserts in batches, triggers index build + compaction
`compact_and_watch.py`	Polls Milvus for compaction state, waits for segment consolidation, monitors index build progress
`enhanced_bench.py`	Unified benchmark: Path A (timed/query-count, per-worker CSV, P99.99) + Path B (sweep, warm/cold, budget, memory estimator)
`simple_bench.py`	Simplified benchmark variant (subset of Path A features)
`collection_mgr.py`	Collection CRUD, schema introspection, index parameter detection
`config_loader.py`	YAML loading with CLI override merge logic
`list_collections.py`	tabulate-based collection info display

5. System Requirements

5.1 Minimum

- CPU: 4+ cores | RAM: 16 GB | Storage: 20 GB free SSD
- OS: Linux (Ubuntu 22.04+) | Python: 3.10+ | Docker & Docker Compose
- No GPU required

5.2 Recommended

- CPU: 16+ cores | RAM: 64–128 GB | Storage: 200+ GB NVMe (PCIe Gen4+)
- pandas for full P99.99 statistics, tabulate for collection display

5.3 Memory Requirements by Configuration

Config	Vecs	Dim	Index	Vec Data	Index OH	HNSW RAM	DiskANN RAM
Smoke	100K	128	DiskANN	~49 MB	~6 MB	N/A	~55 MB
Light	500K	512	DiskANN	~977 MB	~32 MB	N/A	~1 GB
Medium	1M	1536	DiskANN	~5.7 GB	~64 MB	N/A	~5.8 GB (SSD)
Medium	1M	1536	HNSW	~5.7 GB	~1 GB	~6.7 GB	N/A
Heavy	10M	1536	DiskANN	~57 GB	~640 MB	N/A	~58 GB (SSD)
Heavy	10M	1536	HNSW	~57 GB	~10 GB	~67 GB	N/A

> *DiskANN RAM requirements are dramatically lower than HNSW because the graph lives on SSD. The bulk of data is read from SSD during search.*

Use the built-in memory estimator to plan before loading:

```
python vdbbench/enhanced_bench.py --estimate-only --est-index-type HNSW --est-n 10000000
--est-dim 1536 --est-hnsw-m 64
```

6. Installation

```
git clone https://github.com/mlcommons/storage.git
cd storage/vdb_benchmark
pip install -e ./
```

Manual dependency install: pip install pymilvus numpy pyyaml tabulate pandas

Verify: python vdbbench/enhanced_bench.py --help | mlpstorage vectordb run --help

7. Deploying Milvus

The benchmark requires a running Milvus instance. Docker Compose stacks are provided.

Local Storage (MinIO) — Recommended for SSD Testing

```
cp stacks/milvus/standalone/minio/.env.example stacks/milvus/standalone/minio/.env
# Edit .env: Set MILVUS_DATA_DIR, MINIO_DATA_DIR, ETCD_DATA_DIR to target NVMe mount
docker compose -f stacks/milvus/standalone/minio/docker-compose.yml up -d
```

The stack creates three containers: Milvus v2.5.10 (gRPC :19530), MinIO (S3 object storage :9000), and etcd v3.5.18 (metadata :2379).

NVMe Targeting

Set DOCKER_VOLUME_DIRECTORY in .env to point at your target NVMe mount (e.g., /mnt/nvme0). To compare storage solutions: bring containers down, update mount point in .env, restart. Or use separate compose stacks with different port mappings for parallel testing.

8. Configuration Reference

8.1 YAML Configuration Files

Config File	Vecs	Dim	Index	Shards	Key Build Params
`10m_diskann.yaml`	10M	1536	DiskANN	10	`max_degree=64, sl=200`
`10m_hnsw.yaml`	10M	1536	HNSW	10	`M=64, ef_c=200`
`1m_diskann.yaml`	1M	1536	DiskANN	1	`max_degree=64, sl=200`
`1m_hnsw.yaml`	1M	1536	HNSW	1	`M=64, ef_c=200`
`1m_diskann_512dim.yaml`	1M	512	DiskANN	1	`max_degree=32, sl=100`
`1m_aisaq_512dim.yaml`	1M	512	AISAQ	1	`inline_pq=32, md=32, sl=100`

All configs use COSINE metric, FLOAT_VECTOR dtype. Distributions: uniform (default), normal, zipfian (v3.0).

8.2 CLI Parameters — Path A (Runtime / Query-Count)

Parameter	Default	Description
`--collection`	required	ANN-indexed collection name
`--runtime`	None	Benchmark duration in seconds
`--queries`	1000	Total query count
`--batch-size`	required	Queries per batch
`--processes`	8	Worker processes
`--search-limit`	10	Top-k results per query
`--search-ef`	200	ef (HNSW) / search_list (DiskANN, AISAQ) override
`--auto-create-flat`	False	Auto-create FLAT GT collection from source
`--gt-collection`	<coll>_flat_gt	FLAT ground truth collection name
`--output-dir`	vdbbench_results/<ts>	Output directory for CSV + JSON files
`--config`	None	YAML config file (CLI flags override YAML)

8.3 CLI Parameters — Path B (Sweep / Cache / Budget)

Parameter	Default	Description
`--mode`	both	single / mp / both
`--sweep`	False	Enable parameter sweep
`--target-recall`	0.95	Single recall target
`--recall-targets`	None	Multiple targets (e.g., 0.90 0.95 0.99)
`--optimize`	quality	Sweep objective: quality / latency / cost
`--cache-state`	both	warm / cold / both
`--drop-caches-cmd`	sync && echo 3 > ...	Command to drop OS page cache for cold runs
`--restart-milvus-cmd`	None	Optional Milvus restart command
`--milvus-container`	None	Container name(s) for RSS measurement
`--disk-dev`	None	Block device(s) to track (default: all real disks)

`--mem-budget-gb`	None	Max container RSS budget in GB
`--host-mem-reserve-gb`	None	Min host MemAvailable required

9. Quick Start

Run a basic 2-minute storage stress test with DiskANN on 1M vectors:

```
# Step 1: Load vectors
python vdbbench/load_vdb.py --config vdbbench/configs/1m_diskann.yaml

# Step 2: Create FLAT ground truth (one-time)
python vdbbench/enhanced_bench.py --collection mlps_1m_1shards_1536dim_uniform_diskann \
  --auto-create-flat --runtime 1 --batch-size 1 --processes 1

# Step 3: Run benchmark
python vdbbench/enhanced_bench.py --collection mlps_1m_1shards_1536dim_uniform_diskann \
  --runtime 120 --batch-size 10 --processes 4 --search-ef 200
```

10. Running the Benchmark

Scenario 1: Storage-Only Baseline (DiskANN, Warm Cache)

```
python vdbbench/enhanced_bench.py --config vdbbench/configs/10m_diskann.yaml \
  --runtime 300 --batch-size 10 --processes 8 --search-ef 200
```

Scenario 2: Maximum SSD Stress (DiskANN, Cold Cache)

```
python vdbbench/enhanced_bench.py --config vdbbench/configs/10m_diskann.yaml \
  --runtime 300 --batch-size 10 --processes 8 --search-ef 200 --cache-state cold
```

Scenario 3: Peak IOPS Hunt (16 Processes, Sweep)

```
python vdbbench/enhanced_bench.py --collection mlps_10m_...diskann \
  --sweep --target-recall 0.95 --cache-state cold --mode both --processes 16
```

Scenario 4: Recall vs. IOPS Tradeoff Curve

```
python vdbbench/enhanced_bench.py --collection mlps_10m_...diskann \
  --sweep --recall-targets 0.90 0.95 0.99 --optimize latency --cache-state both
```

Using the mlpstorage CLI (v3.0)

```
mlpstorage vectordb datagen --config 10m_diskann --host 127.0.0.1 --num-vectors 10000000
mlpstorage vectordb run --config 10m_diskann --host 127.0.0.1 --num-query-processes 8 --
runtime 300
```

11. Indexing Algorithms and Storage I/O Tradeoffs

Property	DiskANN	HNSW	AISAQ
Index Storage	SSD-resident graph	In-memory graph	Compressed SSD graph
Memory Footprint	Low (PQ + metadata)	High (full graph in RAM)	Low (inline PQ codes)
Search I/O	Random SSD reads (4–64 KB)	Memory-only (no disk I/O)	Random SSD reads + PQ decode
Build Params	max_degree, search_list_size	M, ef_construction	inline_pq, max_degree, search_list_size
Search Knob	search_list (10–4000)	ef (10–4096)	search_list (10–3000)
Storage Stress	HIGH — heavy random reads	LOW — memory-bound	MEDIUM — compressed reads
Best For	Large-scale SSD characterization	Low-latency RAM-rich systems	Memory-constrained + SSD

> *Key Insight: DiskANN is the primary storage stress driver. Use HNSW as a control to isolate storage overhead from query engine overhead.*

12. Vector Dimension Profiles

Dim	Bytes/Vec	1M Vectors	10M Vectors	Typical Embedding Model
128	512 B	~49 MB	~4.8 GB	Lightweight classifiers
512	2,048 B	~1.9 GB	~19.1 GB	e5-small, MiniLM, CLIP-ViT-B
768	3,072 B	~2.9 GB	~28.6 GB	BERT-base, Sentence-BERT
1536	6,144 B	~5.7 GB	~57.2 GB	OpenAI ada-002, Cohere embed-v3

13. Storage Stress Intensity Matrix

Tier	Vectors	Dim	Index	Procs	Search Param	Disk Stress	Dataset
Smoke	100K	128	**DiskANN**	1	`sl=50`	**Minimal**	~50 MB
Light	500K	512	**DiskANN**	2	`sl=100`	**Low**	~1 GB
Medium	1M	1536	**DiskANN**	4	`sl=200`	**Medium**	~5.7 GB
Medium	1M	1536	**HNSW**	4	`ef=128`	**Low (RAM)**	~5.7 GB
Medium	1M	512	**AISAQ**	4	`sl=100`	**Medium**	~2 GB
Heavy	10M	1536	**DiskANN**	8	`sl=200`	**High**	~57 GB
Heavy	10M	1536	**HNSW**	8	`ef=256`	**Low (RAM)**	~57 GB
Extreme	10M	1536	**DiskANN**	16	`sl=800-4000`	**Very High**	~57 GB

14. Complete Test Matrix (T1–T10)

ID	Config	Vecs	Dim	Index	Procs	Cache	Mode	Goal
T1	`1m_diskann`	1M	1536	**DiskANN**	4	warm	Path A 120s	Baseline throughput
T2	`1m_diskann`	1M	1536	**DiskANN**	4	cold	Path A 120s	SSD random read stress
T3	`1m_hnsw`	1M	1536	**HNSW**	4	warm	Path A 120s	In-RAM baseline (control)
T4	`1m_diskann_512d`	1M	512	**DiskANN**	4	both	Path B sweep	Dimension effect on I/O
T5	`1m_aisaq_512d`	1M	512	**AISAQ**	4	both	Path B sweep	AISAQ vs DiskANN I/O
T6	`10m_diskann`	10M	1536	**DiskANN**	8	warm	Path A 300s	Scale throughput
T7	`10m_diskann`	10M	1536	**DiskANN**	8	cold	Path A 300s	Maximum SSD stress
T8	`10m_diskann`	10M	1536	**DiskANN**	16	cold	Path B sweep	Peak IOPS hunt
T9	`10m_hnsw`	10M	1536	**HNSW**	8	warm	Path A 300s	RAM-bound at scale
T10	`10m_diskann`	10M	1536	**DiskANN**	8	both	Path B multi	Recall vs IOPS curve

15. Search Parameter Sweep Grids

When --sweep is enabled, enhanced_bench.py auto-generates search parameter candidates:

Index	Parameter	Sweep Values
DiskANN	`search_list`	10, 20, 50, 100, 200, 400, 800, 1200, 1600, 2000, 2500, 3000, 4000
HNSW	`ef`	10, 16, 32, 64, 128, 256, 512, 1024, 1536, 2048, 3072, 4096
AISAQ	`search_list`	10, 20, 50, 100, 200, 400, 800, 1200, 1600, 2000, 2500, 3000
IVF	`nprobe`	1, 2, 4, 8, 16, 32, 64, 128

16. Warm vs. Cold Cache — SSD Stress Isolation

Cache State	Flag	Behavior	Best For
Warm	`--cache-state warm`	OS page cache absorbs repeated reads	Baseline throughput, recall tuning
Cold	`--cache-state cold`	Drops page cache before each run	NVMe characterization, worst-case latency
Both	`--cache-state both`	Runs warm then cold automatically	Complete characterization

> *For maximum SSD stress: --cache-state cold + DiskANN + --processes 16 + search_list 800+. Use --restart-milvus-cmd to flush internal caches.*

17. Understanding Results

17.1 Key Metrics

Metric	Description
`mean_latency_ms / p95 / p99 / p999 / p9999`	Query latency percentiles in milliseconds
`throughput_qps`	Queries per second (sustained)
`mean_recall / median / p95 / p99`	Recall@k distribution across evaluated queries
`disk_read_mbps / disk_write_mbps`	Average read/write throughput in MB/s
`disk_read_iops / disk_write_iops`	Average read/write IOPS
`read_bytes_per_query`	Storage I/O amplification per query

17.2 Output Files — Path A

- config.json — Run configuration snapshot
- milvus_benchmark_p0..pN.csv — Per-worker timing timeseries
- recall_hits_p0..pN.jsonl — ANN result IDs per query (for recall calculation)
- recall_stats.json — Full recall@k distribution
- statistics.json — Aggregated latency + recall + disk I/O

17.3 Output Files — Path B

- combined_bench_<tag>_<ts>.json — All run results + sweep data
- combined_bench_<tag>_<ts>.csv — Per-run tabular summary
- combined_bench_<tag>_<ts>.sweep.csv — Per-candidate sweep details
- gt_cache/gt_<hash>.npz + .meta.json — Cached ground truth

18. Unit Testing

```
cd tests && make test
# Or: pip install -r requirements.txt && pytest tests/ -v
```

Tests cover configuration loading, vector generation, index management, load_vdb, simple_bench, compaction monitoring, and database connection handling. Tests use mock Milvus connections and do not require a running Milvus instance.

19. Troubleshooting

Load takes too long: 10M vector loads can take up to 8 hours. Use --num-vectors 500000 for quick subset testing, or use 1M configs (~45 min).

Recall shows 0.000: Ensure the FLAT GT collection exists. Re-run with --auto-create-flat. Verify recall_hits_p*.jsonl files are non-empty.

Cold cache same as warm: The drop_caches command requires root. Use sudo or run benchmark as root.

Results vary between runs: Use identical --seed values. Ground truth cache (gt_cache/) ensures consistent recall baselines.

Docker containers not healthy: Check: docker logs milvus-standalone. Common issue: insufficient disk space on Docker volume directory.

References

1. Atlarge Research, TU Delft (2025). “Storage-Based Approximate Nearest Neighbor Search” — I/O characterization of Milvus-DiskANN. IISWC 2025.
2. Shim et al. (2025). “Turbocharging Vector Databases using Modern SSDs” — pgvector SSD optimization with io_uring. VLDB 2025.
3. Dell Technologies (2024). “Vector Database Infrastructure Requirements” — Enterprise VDB infrastructure white paper.
4. Yan et al. (2022). “Manu: A Cloud Native Vector Database Management System” — Milvus architecture. VLDB 2022.
5. Jayaram Subramanya et al. (2019). “DiskANN: Fast Accurate Billion-Point Nearest Neighbor Search on a Single Node”. NeurIPS 2019.
6. Chen (2024). “Characterizing the Dilemma of Performance and Index Size in Billion-Scale Vector Search”. arXiv 2405.03267.